Step 1 – Single-Field Index

Mongosh

Use university

db.createCollection("students")

db.students.insertMany([
{ name: "Alice Smith", age: 20, major: "Computer Science", gpa: 3.8, courses: ["CS101", "MATH201"] },
{ name: "Bob Johnson", age: 22, major: "Engineering", gpa: 3.5, courses: ["ENGR101", "PHYS201"] },
{ name: "Charlie Brown", age: 21, major: "Physics", gpa: 3.9, courses: ["PHYS101", "MATH201"] },
{ name: "David Lee", age: 19, major: "Computer Science", gpa: 3.7, courses: ["CS101", "MATH101"] },
{ name: "Eve Wilson", age: 23, major: "Biology", gpa: 3.6, courses: ["BIO101", "CHEM201"] },
{ name: "Alice Johnson", age: 20, major: "Engineering", gpa: 3.4, courses: ["ENGR101", "MATH101"] },
{ name: "Bob Smith", age: 21, major: "Physics", gpa: 3.8, courses: ["PHYS101", "CS101"] },
{ name: "Charlie Lee", age: 22, major: "Biology", gpa: 3.5, courses: ["BIO101", "PHYS201"] },
{ name: "David Brown", age: 19, major: "Computer Science", gpa: 4.0, courses: ["CS101", "ENGR101"] },
{ name: "Eve Johnson", age: 23, major: "Engineering", gpa: 3.9, courses: ["ENGR101", "CHEM201"] }
])

db.students.find().pretty()

db.students.createIndex({ age: 1 })

Step 2: Analyze Query Execution Plan without Index

db.students.find({ age: 20 }).xplain("executionStats")

```
university> db.students.find({ age: 20 }).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'university.students',
    parsedQuery: { age: { '$eq': 20 } },
    indexFilterSet: false,
    queryHash: '0637AF1A',
    planCacheShapeHash: '0637AF1A',
    planCacheKey: 'D54E87D9',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: { age: 1 },
        indexName: 'age_1',
        isMultiKey: false,
        multiKeyPaths: { age: [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: { age: [ '[20, 20]' ] }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 2,
    executionTimeMillis: 0,
    totalKeysExamined: 2,
    totalDocsExamined: 2,
    executionStages: {
      isCached: false,
      stage: 'FETCH',
      nReturned: 2,
      executionTimeMillisEstimate: 0,
      works: 3,
      advanced: 2,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      docsExamined: 2,
      alreadyHasObj: 0,
      inputStage: {
        stage: 'IXSCAN',
        nReturned: 2,
        executionTimeMillisEstimate: 0,
        works: 3,
        advanced: 2,
        needTime: 0,
        needYield: 0,
        saveState: 0,
        restoreState: 0,
        isEOF: 1,
        keyPattern: { age: 1 },
        indexName: 'age_1',
        isMultiKey: false,
        multiKeyPaths: { age: [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: { age: [ '[20, 20]' ] },
        keysExamined: 2,
        seeks: 1,
        dupsTested: 0,
        dupsDropped: 0
      }
```

```
  },
  queryShapeHash: 'ABCB17C9C0BEB9A543AA17AF6312D112544B44F4190E6E4CB0CCBD77380C8387',
  command: { find: 'students', filter: { age: 20 }, '$db': 'university' },
  serverInfo: {
    host: 'Richard',
    port: 27017,
    version: '8.2.6',
    gitVersion: '5d25c835745d06f712320b6cdae9d50b7b43663e'
  },
  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFrameworkControl: 'trySbeRestricted',
    internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
  },
  ok: 1
}
```

Step 3: Analyze Query Execution Plan with Index

Command: db.students.find({ age: 20 }).explain("executionStats")

```
university> db.students.find({ age: 20 }).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'university.students',
    parsedQuery: { age: { '$eq': 20 } },
    indexFilterSet: false,
    queryHash: '0637AF1A',
    planCacheShapeHash: '0637AF1A',
    planCacheKey: 'D54E87D9',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: { age: 1 },
        indexName: 'age_1',
        isMultiKey: false,
        multiKeyPaths: { age: [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: { age: [ '[20, 20]' ] }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 2,
    executionTimeMillis: 0,
    totalKeysExamined: 2,
    totalDocsExamined: 2,
    executionStages: {
      isCached: false,
      stage: 'FETCH',
      nReturned: 2,
      executionTimeMillisEstimate: 0,
      works: 3,
      advanced: 2,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      docsExamined: 2,
      alreadyHasObj: 0,
      inputStage: {
        stage: 'IXSCAN',
        nReturned: 2,
        executionTimeMillisEstimate: 0,
        works: 3,
        advanced: 2,
        needTime: 0,
        needYield: 0,
        saveState: 0,
        restoreState: 0,
        isEOF: 1,
        keyPattern: { age: 1 },
        indexName: 'age_1',
        isMultiKey: false,
        multiKeyPaths: { age: [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: { age: [ '[20, 20]' ] },
        keysExamined: 2,
        seeks: 1,
        dupsTested: 0,
        dupsDropped: 0
      }
```

```
  },
  queryShapeHash: 'ABCB17C9C0BEB9A543AA17AF6312D112544B44F4190E6E4CB0CCBD77380C8387',
  command: { find: 'students', filter: { age: 20 }, '$db': 'university' },
  serverInfo: {
    host: 'Richard',
    port: 27017,
    version: '8.2.6',
    gitVersion: '5d25c835745d06f712320b6cdae9d50b7b43663e'
  },
  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFrameworkControl: 'trySbeRestricted',
    internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
  },
  ok: 1
}
```

Step 4: Create a Compound Index

Command: db.students.createIndex({ major: 1, gpa: -1 })

```
university> db.students.createIndex({ major: 1, gpa: -1 })
major_1_gpa_-1
```

Step 5: Analyze Query Execution Plan with Compound Index

Command: db.students.find({ major: "Computer Science" }).sort({ gpa: -1 }).explain("executionStats")

```
university>  db.students.find( { major: "Computer Science" } ).sort( { gpa: -1 } ).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'university.students',
    parsedQuery: { major: { '$eq': 'Computer Science' } },
    indexFilterSet: false,
    queryHash: 'ECE62D24',
    planCacheShapeHash: 'ECE62D24',
    planCacheKey: '7D937067',
    optimizationTimeMillis: 1,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: { major: 1, gpa: -1 },
        indexName: 'major_1_gpa_-1',
        isMultiKey: false,
        multiKeyPaths: { major: [], gpa: [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: {
          major: [ '["Computer Science", "Computer Science"]' ],
          gpa: [ '[MaxKey, MinKey]' ]
        }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 3,
    executionTimeMillis: 1,
```

```
    totalKeysExamined: 3,
    totalDocsExamined: 3,
    executionStages: {
      isCached: false,
      stage: 'FETCH',
      nReturned: 3,
      executionTimeMillisEstimate: 0,
      works: 4,
      advanced: 3,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      docsExamined: 3,
      alreadyHasObj: 0,
      inputStage: {
        stage: 'IXSCAN',
        nReturned: 3,
        executionTimeMillisEstimate: 0,
        works: 4,
        advanced: 3,
        needTime: 0,
        needYield: 0,
        saveState: 0,
        restoreState: 0,
        isEOF: 1,
        keyPattern: { major: 1, gpa: -1 },
        indexName: 'major_1_gpa_-1',
        isMultiKey: false,
        multiKeyPaths: { major: [], gpa: [] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: {
          major: [ '["Computer Science", "Computer Science"]' ],
          gpa: [ '[MaxKey, MinKey]' ]
        },
        keysExamined: 3,
```

```
        seeks: 1,
        dupsTested: 0,
        dupsDropped: 0
      }
    }
  },
  queryShapeHash: '721EC999FC25193AFDB60F95F8C2413FDC31212425B0FE698305E16FACCB56B1',
  command: {
    find: 'students',
    filter: { major: 'Computer Science' },
    sort: { gpa: -1 },
    '$db': 'university'
  },
  serverInfo: {
    host: 'Richard',
    port: 27017,
    version: '8.2.6',
    gitVersion: '5d25c835745d06f712320b6cdae9d50b7b43663e'
  },
  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFrameworkControl: 'trySbeRestricted',
    internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
  },
  ok: 1
}
```

Step 6: Create a Multi-Key Index

Command: db.students.createIndex({ courses: 1 })

```
university>  db.students.createIndex( { courses: 1 } )
courses_1
```

Step 7: Analyze Query Execution Plan with Multi-Key Index

Command: db.students.find({ courses: "CS101" }).explain("executionStats")

```
university>  db.students.find( { courses: "CS101" } ).explain("executionStats")
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'university.students',
    parsedQuery: { courses: { '$eq': 'CS101' } },
    indexFilterSet: false,
    queryHash: '69CA0877',
    planCacheShapeHash: '69CA0877',
    planCacheKey: '285DD790',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'FETCH',
      inputStage: {
        stage: 'IXSCAN',
        keyPattern: { courses: 1 },
        indexName: 'courses_1',
        isMultiKey: true,
        multiKeyPaths: { courses: [ 'courses' ] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: { courses: [ '["CS101", "CS101"]' ] }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 0,
    totalKeysExamined: 4,
    totalDocsExamined: 4,
```

```
    executionStages: {
      isCached: false,
      stage: 'FETCH',
      nReturned: 4,
      executionTimeMillisEstimate: 0,
      works: 5,
      advanced: 4,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      docsExamined: 4,
      alreadyHasObj: 0,
      inputStage: {
        stage: 'IXSCAN',
        nReturned: 4,
        executionTimeMillisEstimate: 0,
        works: 5,
        advanced: 4,
        needTime: 0,
        needYield: 0,
        saveState: 0,
        restoreState: 0,
        isEOF: 1,
        keyPattern: { courses: 1 },
        indexName: 'courses_1',
        isMultiKey: true,
        multiKeyPaths: { courses: [ 'courses' ] },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: { courses: [ '["CS101", "CS101"]' ] },
        keysExamined: 4,
        seeks: 1,
        dupsTested: 4,
        dupsDropped: 0
      }
```

```
  },
  queryShapeHash: '21E645F7646F91E9E554269E30096F16F5358F82CAE8ACC00825153891AF64EA',
  command: {
    find: 'students',
    filter: { courses: 'CS101' },
    '$db': 'university'
  },
  serverInfo: {
    host: 'Richard',
    port: 27017,
    version: '8.2.6',
    gitVersion: '5d25c835745d06f712320b6cdae9d50b7b43663e'
  },
  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFrameworkControl: 'trySbeRestricted',
    internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
  },
  ok: 1
}
```

Step 8: Drop an Index

Command: db.students.getIndexes()

```
university>  db.students.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { age: 1 }, name: 'age_1' },
  { v: 2, key: { major: 1, gpa: -1 }, name: 'major_1_gpa_-1' },
  { v: 2, key: { courses: 1 }, name: 'courses_1' }
]
```

Command: db.students.dropIndex("age_1")

```
university>  db.students.dropIndex("age_1")
{ nIndexesWas: 4, ok: 1 }
```

Step 9: Drop All Indexes (Except the Default _id Index)

Command: db.students.dropIndexes()

```
university>  db.students.dropIndexes()
{
  nIndexesWas: 3,
  msg: 'non-_id indexes dropped for collection',
  ok: 1
}
```

Summary of Observations the Impact of Indexes on Query Performance

In this lab, we analyzed query performance before and after using indexes in MongoDB. Initially, queries executed without indexes used the COLLSCAN method, which reads all documents in the collection. This resulted in a large number of documents being examined, making it less efficient, especially for large datasets.

After creating an index on a specific field, such as age, MongoDB used IXSCAN (Index Scan). This significantly reduced the number of documents examined, as the database directly accessed the relevant data through the index. This resulted in faster and more efficient query performance.

Furthermore, using compound indexes allows MongoDB to optimize queries that involve simultaneous filtering and sorting, such as those for fields like major and gpa. Multi-key indexes are used for array fields like courses, speeding up element searches within the array.

Overall, indexes significantly improve query performance, especially for large datasets. However, the use of indexes requires careful consideration, as they can increase storage usage and slow down insert/update operations.